

EXAME MODELO - BASES DE DADOS

Faculdade de Ciências da Universidade de Lisboa

Duração: 2 horas

INSTRUÇÕES

- O exame tem a duração de 2 horas (120 minutos).
- O exame é sem consulta, mas inclui um glossário e formulário em anexo.
- Responda de forma clara e objetiva. Privilegie a aplicação de conceitos em vez de definições memorizadas.
- A interpretação correta das questões faz parte da avaliação.
- Leia atentamente todo o enunciado antes de começar.

CENÁRIO BASE

A empresa **TravelEasy** opera uma plataforma online de reservas de viagens que integra voos, hotéis e aluguer de carros. O sistema gere milhões de utilizadores, milhares de fornecedores (companhias aéreas, hotéis, empresas de rent-a-car) e processa cerca de 100.000 reservas por dia.

Contexto técnico atual:

- Base de dados relacional PostgreSQL (servidor único)
- Picos de utilização: 15h-18h (horário europeu) e 9h-12h (horário americano)
- 80% das consultas são pesquisas (read-heavy workload)
- 20% são transações de reserva e pagamento
- Dados históricos: 5 anos de reservas (aprox. 180 milhões de registos)
- Equipa de business intelligence precisa de relatórios analíticos diários

QUESTÕES

Questão 1 (3 valores)

A TravelEasy pretende modelar a seguinte funcionalidade:

"Um utilizador pode fazer várias reservas. Cada reserva pode incluir um voo, um hotel, e/ou um carro. Para voos, é necessário guardar companhia aérea, número do voo, aeroporto de origem, aeroporto de destino, data/hora de partida e chegada, e classe (económica, executiva, primeira). Para hotéis, guarda-se nome do hotel, localização, tipo de quarto, número de noites, e regime de refeições. Para carros, regista-se empresa, modelo do carro, local de levantamento, local de devolução, e período de aluguer."

a) (2 valores) Desenhe um **diagrama Entidade-Associação (ER)** que modele este cenário. Indique claramente:

- Entidades e seus atributos principais
- Relações entre entidades e suas cardinalidades
- Chaves primárias (sublinhadas)

b) (1 valor) Identifique **uma decisão de modelação** que tomou e justifique-a brevemente. Por exemplo: escolheu usar generalização/especialização? Criou uma entidade associativa? Porquê?

Questão 2 (3 valores)

Considere o seguinte esquema relacional resultante da conversão do modelo ER:

```
Utilizador(id_utilizador, nome, email, telefone, data_registo)
Reserva(id_reserva, id_utilizador*, data_reserva, preco_total, estado)
ItemReserva(id_item, id_reserva*, tipo_item, dados_json)
Pagamento(id_pagamento, id_reserva*, metodo, valor, data_pagamento)
```

Foi executada a seguinte query SQL:

```
SELECT u.nome, u.email, COUNT(r.id_reserva) as total_reservas,
       SUM(r.preco_total) as gasto_total
FROM Utilizador u
LEFT JOIN Reserva r ON u.id_utilizador = r.id_utilizador
WHERE r.data_reserva >= '2024-01-01'
GROUP BY u.id_utilizador, u.nome, u.email
HAVING SUM(r.preco_total) > 1000
ORDER BY gasto_total DESC
LIMIT 100;
```

a) (1 valor) **Interprete esta query:** O que ela retorna? Explique o papel de cada cláusula principal (WHERE, GROUP BY, HAVING, ORDER BY, LIMIT).

b) (1 valor) Um utilizador reclamou que **não aparece nos resultados** apesar de ter gasto 1200€ em reservas durante 2024. Identifique o **problema na query** que pode causar esta situação e sugira a correção.

c) (1 valor) A query está a demorar muito tempo a executar. Sugira **dois índices** que poderiam melhorar a performance e explique porquê.

Questão 3 (3 valores)

A tabela `Reserva` começou com o seguinte design:

```
Reserva_v1(
    id_reserva,
    id_utilizador,
    nome_utilizador,      -- nome do utilizador
    email_utilizador,     -- email do utilizador
```

```

telefone_utilizador, -- telefone do utilizador
data_reserva,
voo_companhia,        -- pode ser NULL
voo_numero,          -- pode ser NULL
hotel_nome,          -- pode ser NULL
hotel_localizacao,   -- pode ser NULL
carro_empresa,       -- pode ser NULL
carro_modelo,        -- pode ser NULL
preco_total,
estado
)

```

a) (1.5 valores) Identifique **três problemas específicos** com este esquema:

- **Redundância de dados:** Que dados estão a ser duplicados desnecessariamente? Qual o impacto em termos de espaço de armazenamento?
- **Anomalias de atualização:** Dê um exemplo concreto de um problema que pode ocorrer quando se atualiza informação nesta tabela.
- **Anomalias de inserção ou eliminação:** Dê um exemplo concreto de uma limitação ou inconsistência que pode surgir.

b) (1.5 valores) O esquema foi redesenhado para o seguinte:

```

Utilizador(id_utilizador, nome, email, telefone, data_registo)
Reserva(id_reserva, id_utilizador*, data_reserva, preco_total, estado)
ItemVoo(id_item, id_reserva*, companhia, numero, origem, destino, ...)
ItemHotel(id_item, id_reserva*, nome_hotel, localizacao, ...)
ItemCarro(id_item, id_reserva*, empresa, modelo, ...)

```

Explique **por que razão** esta organização é melhor, considerando que:

- O sistema tem milhões de utilizadores
- Cada utilizador pode fazer dezenas de reservas
- Os dados dos utilizadores mudam ocasionalmente (ex: novo email, novo telefone)
- Nem todas as reservas incluem voo, hotel e carro simultaneamente

Questão 4 (3 valores)

A TravelEasy está a enfrentar problemas de **concorrência** durante os períodos de pico. Considere o seguinte cenário:

Situação: Um voo para Paris tem apenas 2 lugares disponíveis em classe executiva. Dois utilizadores (Ana e Bruno) estão a tentar reservar simultaneamente:

Transação T1 (Ana):

1. SELECT lugares_disponiveis

Transação T2 (Bruno):

1. SELECT lugares_disponiveis

```
FROM Voo WHERE id=747
--> resultado: 2
```

```
FROM Voo WHERE id=747
--> resultado: 2
```

2. [Ana confirma reserva]

2. [Bruno confirma reserva]

3. INSERT INTO Reserva ...

3. INSERT INTO Reserva ...

4. UPDATE Voo
SET lugares_disponiveis = 1
WHERE id=747

4. UPDATE Voo
SET lugares_disponiveis = 1
WHERE id=747

5. COMMIT

5. COMMIT

a) (1 valor) Identifique o **problema de concorrência** que pode ocorrer nesta situação e explique as suas **consequências práticas** para o negócio da TravelEasy.

b) (1 valor) Sugira **duas abordagens diferentes** para resolver este problema, explicando brevemente como cada uma funcionaria.

c) (1 valor) Se a TravelEasy usar o **nível de isolamento READ COMMITTED**, o problema seria resolvido? Justifique. Que nível de isolamento seria mais adequado?

Questão 5 (3.5 valores)

A equipa técnica está a considerar introduzir **MongoDB** (base de dados NoSQL orientada a documentos) para parte do sistema.

Considere os seguintes requisitos:

R1: Armazenar o histórico de pesquisas dos utilizadores (queries de voos/hotéis) para personalização

R2: Gestão de pagamentos e faturação (requer transações ACID)

R3: Catálogo de hotéis com reviews dos utilizadores (estrutura variável: alguns hotéis têm fotos, vídeos, amenities diferentes)

R4: Sistema de notificações em tempo real (alertas de preço, confirmações)

a) (2 valores) Para **cada requisito (R1 a R4)**, indique se recomendaria **PostgreSQL (relacional)** ou **MongoDB (NoSQL)** e **justifique** a sua escolha com base nas características de cada tecnologia.

b) (1.5 valores) Suponha que decidiram usar MongoDB para armazenar os perfis de utilizador com as suas preferências e histórico. Desenhe a **estrutura de um documento JSON** que represente um utilizador com:

- Dados pessoais básicos
- Preferências de viagem (destinos favoritos, classe preferida)
- Últimas 3 pesquisas realizadas

Explique **uma vantagem** desta abordagem comparada com o modelo relacional.

Questão 6 (2.5 valores)

O diretor de tecnologia da TravelEasy está preocupado com a **escalabilidade** do sistema atual (PostgreSQL num único servidor) e considerando diferentes arquiteturas.

Cenário: O sistema processa atualmente:

- 100.000 reservas/dia
- 80% consultas (pesquisas), 20% escritas (reservas/pagamentos)
- Utilizadores distribuídos globalmente (Europa, América, Ásia)
- Picos concentrados em horários específicos por região

a) (1.5 valores) Compare **duas estratégias de escalabilidade**:

- **Estratégia A:** Replicação com um master (escritas) e múltiplos slaves (leituras)
- **Estratégia B:** Particionamento (sharding) por região geográfica

Para cada estratégia, indique:

- Uma vantagem principal
- Uma desvantagem ou desafio
- Que tipo de workload beneficia mais

b) (1 valor) A equipa de gestão quer **relatórios analíticos** sobre:

- Destinos mais populares por mês
- Receita por companhia aérea/hotel
- Tendências de preços e ocupação

Executar estas queries analíticas complexas na base de dados de produção está a causar lentidão. Que **arquitetura de dados** recomendaria para resolver este problema? Justifique brevemente.

Questão 7 (2 valores)

A TravelEasy decidiu usar MongoDB para armazenar as avaliações (reviews) dos hotéis. A coleção `hotel_reviews` tem documentos com a seguinte estrutura:

```
{
  "_id": ObjectId("..."),
  "hotel_id": "H12345",
  "hotel_name": "Grand Hotel Lisboa",
  "city": "Lisboa",
  "country": "Portugal",
  "reviews": [
    {
      "user_id": "U567",
      "rating": 5,
```

```

        "comment": "Excelente localização!",
        "date": ISODate("2024-11-15")
    },
    {
        "user_id": "U892",
        "rating": 4,
        "comment": "Muito bom, mas caro.",
        "date": ISODate("2024-12-01")
    }
]
}

```

Foi executada a seguinte query MongoDB:

```

db.hotel_reviews.aggregate([
  {
    $match: {
      country: "Portugal",
      "reviews.date": { $gte: ISODate("2024-01-01") }
    }
  },
  {
    $unwind: "$reviews"
  },
  {
    $match: {
      "reviews.date": { $gte: ISODate("2024-01-01") }
    }
  },
  {
    $group: {
      _id: "$hotel_name",
      avg_rating: { $avg: "$reviews.rating" },
      total_reviews: { $sum: 1 }
    }
  },
  {
    $match: {
      total_reviews: { $gte: 10 }
    }
  },
  {
    $sort: { avg_rating: -1 }
  }
]

```

```
    },  
    {  
      $limit: 5  
    }  
  ] )
```

- a)** (1.5 valores) **Interprete esta query:** O que ela retorna? Explique o que faz cada operador do pipeline de agregação (\$match, \$unwind, \$group, \$sort, \$limit).
- b)** (0.5 valores) Por que razão há **dois operadores \$match** na query? Não poderia ser apenas um? Explique.

ANEXO: GLOSSÁRIO

ACID: Atomicidade, Consistência, Isolamento, Durabilidade - propriedades de transações em SGBD relacionais

Agregação (Aggregation): Operação que processa múltiplos documentos e retorna resultados computados (MongoDB)

Cardinalidade: Número de instâncias de uma entidade que podem estar associadas a instâncias de outra entidade (1:1, 1:N, N:M)

Chave Primária (Primary Key): Atributo ou conjunto de atributos que identifica unicamente cada tuplo numa relação

Chave Estrangeira (Foreign Key): Atributo ou conjunto de atributos que referencia uma chave primária noutra tabela

Documento: Estrutura de dados em formato JSON/BSON que armazena informação em bases de dados NoSQL orientadas a documentos

Índice: Estrutura de dados que melhora a velocidade de operações de pesquisa numa tabela de base de dados

NoSQL: Sistemas de gestão de bases de dados não-relacionais (ex: MongoDB, Cassandra, Redis)

Pipeline: Sequência de operações de processamento de dados (usado em agregações MongoDB)

Read-heavy workload: Sistema onde a maioria das operações são leituras (consultas) em vez de escritas

Redundância: Duplicação desnecessária de dados em múltiplos locais da base de dados

Replicação: Técnica de manter cópias de dados em múltiplos servidores para disponibilidade e performance

Serializabilidade: Propriedade de transações concorrentes que garante que o resultado é equivalente a uma execução em série

Sharding (Particionamento): Técnica de dividir dados entre múltiplos servidores baseada em critérios específicos

Transação: Unidade lógica de trabalho que agrupa múltiplas operações de base de dados

ANEXO: FORMULÁRIO

Níveis de Isolamento (do menos ao mais restritivo)

1. **READ UNCOMMITTED:** Permite leituras de dados não committed (dirty reads)
2. **READ COMMITTED:** Apenas lê dados committed
3. **REPEATABLE READ:** Garante leituras consistentes dos mesmos dados
4. **SERIALIZABLE:** Nível mais alto, garante serializabilidade completa

Comandos SQL básicos

```
-- Junções
INNER JOIN    -- apenas tuplos com correspondência em ambas as tabelas
LEFT JOIN     -- todos os tuplos da tabela esquerda
RIGHT JOIN    -- todos os tuplos da tabela direita
FULL JOIN     -- todos os tuplos de ambas as tabelas

-- Agregação
COUNT(), SUM(), AVG(), MIN(), MAX()
GROUP BY      -- agrupa resultados
HAVING        -- filtro após agregação
```

Operadores MongoDB (Aggregation Pipeline)

```
$match        // filtra documentos (equivalente ao WHERE em SQL)
$group        // agrupa documentos por um campo (equivalente ao GROUP BY)
$project      // seleciona/transforma campos a incluir no output
$sort         // ordena resultados (1 = ascendente, -1 = descendente)
$limit        // limita o número de documentos retornados
$unwind       // "desdobra" arrays, criando um documento por elemento
$lookup       // faz join entre coleções

// Operadores de comparação
$eq           // igual a
$gt           // maior que
$gte          // maior ou igual a
$lt           // menor que
$lte          // menor ou igual a
$ne           // diferente de

// Operadores de agregação
$avg          // média
```

```
$sum      // soma  
$min      // mínimo  
$max      // máximo  
$count    // contagem
```

FIM DO EXAME